

OKC Thunder Intern Project Writeup

By Jack Bansen

1. Introduction & Approach

My goal was to predict the probability that an NBA shot attempt is made, given features derived from player data, evaluated using log-loss against a benchmark model

To start I just wanted to look at the data and knew I would have to most likely apply some level of feature engineering. Once I looked at the data I figured a logistic regression (GLM) would be the best way to start as it's easy and standard. I also wanted to do a deeper model so I would ask ChatGPT for deeper models (see markdown). This would lead me to a gradient-boosted tree model (XGBoost) as a deeper and hopefully more accurate model for capturing nonlinear interactions.

- Baseline model had a log-loss of ~ 0.6896
- Logistic regression had a log-loss of 0.6396
- XGBoost had a log-loss of 0.6295

$(0.6896 - 0.6295) / 0.6896 = 0.0601 / 0.6896 \approx 0.0872$ So the model I created was an improvement on the baseline by 8.72% which can be valuable as a tool to know what XGBoost can achieve. Not only that but using the same formula against the GLM it performed 1.5% better, which tells that XGBoost does perform better than logistic regression in this scenario. The baseline model treats every shot identically XGBoost differentiates based on distance, contest level, shot type, and shot clock.

2. Data & Feature Engineering

The training data included 425,719 shot attempts across several seasons, with a base make rate of 45.8%. Feature engineering focused on transforming several raw columns into more model-usable forms:

closestdefapproach arrived as a string ("20.05,18.82,16.57,13.86") representing the closest defender's distance at 1s, 0.75s, 0.5s, and 0.25s before the shot. I parsed this into four numeric columns and derived **closeout_speed**, rate at which defender closed the gap, as a more directly interpretable signal than the raw distances alone.

Contester columns (contester1-4_id, distcont1-4) were wide-format with a variable number of contesters per shot. I found contester4_id and distcont4 were entirely empty and thus dropped them. I reshaped the remaining columns into two summary features: **min_contester_dist** (closest contester's distance) and **contesters_within_4ft** (count of contesters within 4 feet).

min_contester_dist is NA when num_contesters == 0, it's not missing data, but a structural signal that no contester was recorded. I imputed these with closestdefdist (closest defender's distance) as the most relevant available substitute.

3. Baseline Model: Logistic Regression

```

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)    2.5753743   0.0360460   71.447 < 2e-16 ***
distance      -0.0822623   0.0017580  -46.792 < 2e-16 ***
three_num       0.2464794   0.0212132   11.619 < 2e-16 ***
dribblesbefore -0.0085092   0.0011863   -7.173 7.34e-13 ***
shotclock       0.0139081   0.0008682   16.020 < 2e-16 ***
closestdefdist  0.0197375   0.0161289    1.224 0.221053
shooterspeed   -0.0168311   0.0013891  -12.117 < 2e-16 ***
shottypelfloater -1.6485060   0.0296920  -55.520 < 2e-16 ***
shottypelheave  -1.3547571   0.1277822  -10.602 < 2e-16 ***
shottypeljumper -1.3444505   0.0349842  -38.430 < 2e-16 ***
shottypelayup  -1.4851952   0.0236658  -62.757 < 2e-16 ***
shottypelpost  -1.5170899   0.0318615  -47.615 < 2e-16 ***
shottypeltip   -1.9639111   0.0349954  -56.119 < 2e-16 ***
gamestateHALFCOURT -0.0358527   0.0162419   -2.207 0.027285 *
gamestateTRANSITION -0.0645693   0.0183083   -3.527 0.000421 ***
contested_num  -0.2316419   0.0159437  -14.529 < 2e-16 ***
closeout_speed  0.0027390   0.0009133    2.999 0.002707 **
min_contester_dist  0.0040765   0.0159693    0.255 0.798513
contesters_within_4ft -0.4370191   0.0081911  -53.353 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 469756  on 340575  degrees of freedom
Residual deviance: 435949  on 340557  degrees of freedom
AIC: 435987

Number of Fisher Scoring iterations: 4

```

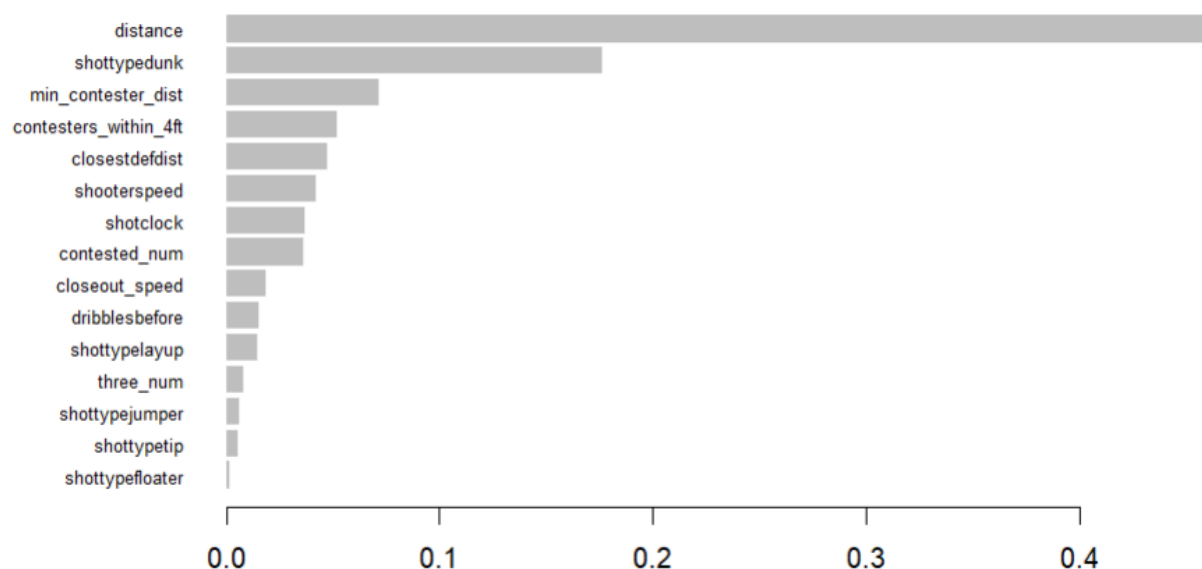
Several coefficients align with basketball intuition: **distance** is strongly negative, all **shottype** levels are negative relative to dunks (reference level), and **contesters_within_4ft** is strongly negative (more defenders nearby reduces shot make probability).

A notable finding came from variance inflation factors (`car::vif(glm_r)`): **closestdefdist** and **min_contester_dist** which had VIF values around 150 meaning there was severe multicollinearity. This explains why neither coefficient was individually significant despite defender proximity clearly mattering overall. The two features carry almost the same information, and the linear model has trouble separating their individual contributions.

The GLM achieved a validation log-loss of **0.6396**, an improvement over the base-rate benchmark of 0.6896

4. Final Model: XGBoost

I moved to XGBoost as my final model because I reviewed and determined that capturing the nonlinear feature interactions the GLM's collinearity issues were obscuring, without requiring me to manually engineer every interaction term was a reasonable and productive model to pursue.



Shot **distance** is by far the strongest predictor, consistent with the GLM baseline. **shottypedunk** ranks second, capturing the near-automatic nature of dunks beyond what distance alone explains. Defender-proximity signals such as **min_contester_dist**, **contesters_within_4ft**, and **closestdefdist** collectively contribute importance but are split across three correlated features rather than concentrated in one which is consistent with the severe multicollinearity. Shockingly, **three_num** ranks low despite being a significant GLM coefficient, likely because distance already implicitly encodes most of this information since three-point attempts are far out shots.

The final XGBoost model reached a validation log-loss of **0.6297**, with a small gap between training and test loss (0.6259 vs. 0.6297 at 401 rounds), indicating the model generalized well and provided no evidence of overfitting at this point in the project.

5. What Didn't Work: Shooter/Defender Skill Features

What I thought would play a major role is shooter and defender identity. I assumed a good shooter's contested 24-footer isn't equally likely to go in as a poor shooter's. I tested this by engineering leave-one-out field goal percentage for both **shooter_id** and **closestdef_id**.

A leakage bug:

My first attempt computed these stats from the full training set before I split them into train and test folds. This meant a shooter's aggregate FG% in the training fold could include information from shots sitting in my validation fold. This ended up inflating my validation log-loss to an unrealistic 0.4236. I went to fix this by computing the statistics only from the training split for validation purposes, while for the actual final submission, computing stats from the entire training set to score the real test set is correct, because the real test outcomes are never touched.

Small-sample noise:

After fixing the leakage, the log-loss value was still worse than if I didn't include these features (0.634 vs. 0.6295). I saw that the 10th percentile of shooter attempt counts was only 14 shots, meaning that many players' raw FG% estimates were really noisy. I applied empirical Bayes shrinkage, pulling each shooter's estimate toward the league mean in proportion to their attempt count. Even with shrinkage strengths up to $k=200$, a head-to-head comparison on identical train/validation splits showed the shooter/defender features still made the validation log-loss worse (0.6357 with vs. 0.6297 without). I kept running into road blocks and concluded that it is either already partially captured by other features like shottype, distance, contested or would require a larger dataset or maybe different treatment to extract reliably. I dropped these features from the final model and noted the failures rather than force a marginal or negative addition.

6. Reflections & Future Improvements

With more time or data, I would try to explore restricting shooter/defender identity features to high-volume players by using a rolling/recent-form shooter features instead

of season-long aggregate FG% and using the mean for everyone else. I would also consider exploring shot zone features derived from specific locations using locationx/locationy instead of raw distance to see if there is any change for things like corner threes or top of the arc threes. Finally I would try and do proper k-fold cross-validation for hyperparameter tuning rather than a single train/validation split; and ensembling the GLM and XGBoost predictions.